

Assignment 4 — CS 331

Software Engineering Lab



Course: CS 331 — Software Engineering Lab | **Assignment 4** (Total Marks = 20)

Project: Argus — Continuous Monitoring & Alert Automation System

I. Software Architecture Style [10 Marks]



Chosen Architecture: Event-Driven Architecture (EDA)

A. How Argus Falls Under EDA — Component Granularity [5 Marks]

EDA is a paradigm where system flow is driven by **events** — state changes produced, detected, and reacted to by decoupled components. Argus follows EDA because agents **emit metric events** → the server **reacts** by evaluating rules → on breach, **alert events fire** → notifications **fan out** asynchronously.

Component Granularity:

- **Event Producers** (*Fine-Grained*) — Single responsibility, lightweight
 - `argus-agent.sh` — Bash script on each server; collects CPU, Memory, Disk, Network metrics and **pushes** to `POST /api/v1/metrics/ingest` every 60s
 - Each agent is **independent** — zero knowledge of downstream processing; can be deployed on any number of servers without backend changes
- **Event Ingestion & Channel** (*Medium-Grained*) — Multiple related concerns, single domain
 - `MetricIngestionController.java` — Receives metric events from agents

- `MetricService.java` — Persists to MySQL, caches in Redis, **publishes** to WebSocket topics
- **WebSocket STOMP** — Pub-sub channel (`/topic/metrics` , `/topic/alerts` , `/topic/server-status`) decouples producers from consumers
- **Event Processors** (*Medium-Grained*) — Specific event types, independent execution
 - `AlertEvaluationService.java` — Scheduled every 60s; evaluates `AlertRule` entities against metrics
 - `HeartbeatMonitorService.java` — Scheduled every 30s; detects offline servers
 - `AlertService.java` — Manages alert lifecycle (ACTIVE → ACKNOWLEDGED → RESOLVED)
 - Each processor runs on its own thread — failure in one does not block the other
- **Event Reactors** (*Coarse-Grained*) — Aggregated responsibilities, multiple event streams
 - `NotificationService.java` — Dispatches email alerts via SMTP with retry logic
 - **React Frontend** — Subscribes to WebSocket topics; reactively updates dashboard from multiple streams

Key EDA Characteristics in Argus: Loose coupling (agents know nothing about backend internals), asynchronous communication (ingestion, evaluation, notification run on independent threads), event-based triggering (alerts fire on threshold breach, not explicit calls), and pub-sub pattern (WebSocket STOMP allows multiple consumers without modifying the publisher).

B. Why EDA is the Best Choice for Argus [5 Marks]

- **Scalability** — Deploy `argus-agent.sh` on any number of servers without modifying the backend; agents are stateless and scale independently. Producers and consumers never bottleneck each other because they communicate via the event channel, not direct calls.

- **Real-Time Responsiveness** — WebSocket STOMP pushes alerts and metrics to the dashboard *instantly* (push-based, not poll-based); when `AlertEvaluationService` detects a breach, the dashboard reflects it in sub-second latency.
- **Maintainability** — Agent (Bash), Backend (Spring Boot), and Frontend (React) are fully independent codebases. Adding a new notification channel (e.g., Slack) means writing one new reactor class — no changes to ingestion or evaluation logic.
- **Performance** — Metric ingestion, rule evaluation, and notification dispatch run **concurrently** on separate `@Scheduled` threads. Redis caches real-time state, avoiding repeated DB queries. Async email dispatch ensures a slow SMTP server never blocks alert processing.
- **Fault Tolerance** — If email fails, `NotificationService` retries (max 3) independently; `HeartbeatMonitorService` and the dashboard continue unaffected. Decoupled components prevent downstream failures from cascading upstream.
- **Extensibility** — New metric types, notification channels, or detection algorithms simply subscribe to existing event streams — zero modification to existing components.

Why not other architectures?

- **Layered** — Tight coupling between layers; changes cascade across layers. Polling-based updates add latency — not suited for real-time monitoring.
 - **Monolithic** — Single deployable unit; can't scale agents and alert engine independently. A crash in one module risks bringing down the entire system.
 - **Microservices** — Synchronous REST calls add network latency at every hop. Requires service discovery, API gateways, circuit breakers — overhead disproportionate to Argus's scope.
 - **SOA** — Heavyweight ESB (Enterprise Service Bus) is over-engineered for a monitoring system. EDA provides the same loose coupling without ESB overhead.
-

EDA Architecture Diagram

II. Application Components [10 Marks]

Overview Diagram

Component List

▼ 1. Agent Layer

- `argus-agent.sh` — Collects 10 metric types (CPU, Memory, Disk, Network, Processes, Load, Uptime) from Linux servers
- `argus-agent-loop.sh` — Continuous monitoring variant

▼ 2. Backend — Controllers

- `AuthController.java` — Register, login, JWT validation
- `ServerController.java` — Server CRUD, agent key management
- `MetricIngestionController.java` — Public metric/heartbeat ingestion endpoint
- `AlertController.java` — Alert rules CRUD, acknowledge/resolve alerts

▼ 3. Backend — Services

- `UserService.java` — User management, role-based access (Admin/User)
- `ServerService.java` — Server registration, status tracking
- `MetricService.java` — Metric persistence, querying, real-time publishing
- `AlertEvaluationService.java` — Scheduled rule evaluation (60s interval)
- `AlertService.java` — Alert lifecycle (ACTIVE → ACKNOWLEDGED → RESOLVED)
- `NotificationService.java` — Email dispatch with retry logic
- `HeartbeatMonitorService.java` — Offline detection (30s interval)
- `JwtService.java` — Token generation & validation

▼ 4. Backend — Entities & Database

- `User` → `users` table (id, username, email, role)
- `Server` → `servers` table (id, name, host, agent_key, status)

- `Metric` → `metrics` table (id, server_id, type, value, timestamp)
- `AlertRule` → `alert_rules` table (id, server_id, condition, threshold, severity)
- `Alert` → `alerts` table (id, severity, status, triggered_at)
- `Notification` → `notifications` table (id, alert_id, channel, retry_count)

▼ 5. Frontend — Pages & Components

- `Dashboard.tsx` — Real-time metrics overview (Recharts)
- `Servers.tsx` / `ServerDetail.tsx` — Server list & individual metrics
- `Alerts.tsx` / `AlertRules.tsx` — Alert management & rule configuration
- `Login.tsx` / `Register.tsx` — Authentication
- `MetricCard`, `AlertCard`, `ServerCard`, `StatusBadge` — Reusable UI components
- `use-realtime.ts` — WebSocket subscription hook (STOMP.js)
- `AuthContext.tsx` — Global auth state
- `api.ts` — API client with JWT interceptor

▼ 6. Infrastructure

- **MySQL 8.0** — Persistent storage (6 tables)
- **Redis 7.0** — Real-time metric caching
- **WebSocket (STOMP + SockJS)** — Pub-sub event channel
- **Spring Mail (SMTP)** — Email notifications with retry

Entity-Relationship Diagram



Total: 40+ components across 6 layers (Agent, Controllers, Services, Entities, Frontend, Infrastructure) — all communicating via event-driven patterns through WebSocket pub-sub, scheduled processors, and async notification dispatch.